SecureShield - RBAC API

Mini Project II Report

Project Overview

SecureShield is a Python Flask REST API that implements Role-Based Access Control (RBAC) using JWT authentication and Flask-Bcrypt password hashing. The system enforces the Principle of Least Privilege with two roles: User and Admin.

What Was Built

- Task 1 - Secure Password Storage: Passwords are hashed with Flask-Bcrypt (auto-salted). Plain text is never stored.
- Task 2 - JWT Issuance: /login returns a signed JWT containing username and role, valid for 30 minutes.
- Task 3 - Token Validation: @token_required decorator verifies the JWT on every protected request.
- Task 4 - Role-Based Routing: GET /profile works for all roles. DELETE /user/<id> is admin-only.
- Task 5 - Token Revocation: POST /logout adds the token to an in-memory blacklist to prevent reuse.
- Task 6 - Defensive Logging: Every 403 Forbidden attempt is logged to security.log with timestamp, user, IP, and route.

Q1: Why Is Salting Necessary to Prevent Rainbow Table Attacks?

A rainbow table is a precomputed database that maps common passwords to their hash values. Without salting, an attacker with a stolen hash can look it up in the table and instantly recover the original password.

Salting adds a unique random string to each password before hashing. Even if two users share the same password, their hashes will be completely different because each has its own salt. The attacker would need a separate rainbow table for every possible salt, which is not feasible.

Flask-Bcrypt handles this automatically: generate_password_hash() creates a new random salt and embeds it in the hash on every call, so the same password always produces a different hash.

Q2: Risks of Storing Sensitive Data Inside a JWT Payload

JWT tokens are Base64-encoded, not encrypted. Anyone who has access to a token can decode the payload and read its contents without knowing the secret key. Only the signature is protected - the payload data is always visible.

Storing sensitive data such as passwords, emails, or personal information in a JWT is dangerous

because:

- The payload is readable by anyone who intercepts or receives the token.
- Tokens stored in localStorage can be stolen via XSS attacks, exposing all payload data.
- JWTs cannot be invalidated before expiry, so leaked sensitive data stays exposed.

In SecureShield, the JWT payload contains only the username and role. Passwords are never included.

Technologies Used

- Flask - Web framework for building the REST API
- PyJWT - JWT token generation and signature validation
- Flask-Bcrypt - Password hashing with automatic salting
- Python standard library: json, logging, os, datetime

API Endpoints

```
POST   /register      - Create a new user account
POST   /login         - Authenticate and receive JWT token
POST   /logout        - Revoke current token  [auth required]
GET    /profile       - View your profile     [user or admin]
DELETE /user/<id>     - Delete a user         [admin only]
GET    /users         - List all users        [admin only]
```

Live Demo Summary (YouTube Video)

1. Register user1 (role: user) and admin1 (role: admin)
2. Login as user1 and receive JWT token
3. GET /profile with user token -> 200 OK
4. DELETE /user/1 with user token -> 403 Forbidden (logged to security.log)
5. Login as admin1 and receive admin JWT token
6. DELETE /user/<id> with admin token -> 200 OK
7. POST /logout -> token is revoked
8. GET /profile with revoked token -> 401 Unauthorized
9. Tamper test: edit JWT role at jwt.io, send modified token -> 401 Invalid token